Republic of the Philippines
POLYTECHNIC UNIVERSITY OF THE PHILIPPINES
SAN PEDRO CAMPUS
PUP, Main Road, Barangay United Bayanihan, City of San Pedro, Laguna

ReserveIT Database Roles and Privileges

Activity 4 Progress Documentation

1. Introduction

This document presents the progress and implementation of Activity 4: Database Users, Roles, Privileges, and Access Control for the ReserveIT project.

The activity focuses on configuring PostgreSQL database roles, assigning appropriate privileges, and testing access restrictions. The main objective is to apply the principle of least privilege, ensuring that each database account receives only the permissions needed for its assigned responsibilities.

2. Project Overview

The project used for this activity is **ReserveIT – Classroom and Material Reservation System**.

ReserveIT is a web-based reservation system developed using Python, Flask, and PostgreSQL. It manages classroom reservations, material requests, user accounts, staff activities, and reservation penalties.

The project repository was cloned from:

Repository:
https://github.com/sev-villanueva/ReserveIT-Classroom-and-Material-Reservation-System-

The database was initialized using the project's setup commands and populated with sample records for testing.

The project documentation reviewed included:

- README.md
- docs/access-requirements.md
- docs/user-roles.md

These documents were used as references when identifying the required database accounts and privileges.

3. Database Inspection

Before implementing access control, the project database was initialized and inspected.

The following setup operations were performed:

- Created the ReserveIT database.
- Initialized the database structure.
- Loaded the sample data.
- Checked the database connection using the application runtime account.
- Reviewed the project's access requirements and user roles.

The database setup was performed using PostgreSQL 18.6.

The project's database objects and access requirements were reviewed before assigning privileges. The detailed database inventory and record counts can be added alongside the corresponding PostgreSQL inspection evidence.

4. Role Creation and Configuration

The ReserveIT project uses separate database accounts for application operations, reporting, database administration, and database ownership.

4.1 Group Roles

Three NOLOGIN group roles were configured:

Group Role	Purpose
app_admin_grp	For database administration and record management
app_runtime_grp	For normal application operations
app_readonly_grp	For read-only reporting and viewing approved data

All three group roles were created with the NOLOGIN attribute. This prevents direct database login using the group roles.

4.2 Login Roles

The following login roles were configured:

Login Role	Assigned Group Role	Purpose
reserveit_app	app_runtime_grp	Application database access
reserveit_reporter	app_readonly_grp	Read-only reporting access
reserveit_dba	app_admin_grp	Database administration
reserveit_owner	Separate owner account	Database object ownership and setup

The PostgreSQL postgres account was used for initial database setup and superuser operations.

4.3 Role Responsibilities

- **Application Runtime:** Performs the database operations required by the application.
- **Reporter:** Reads approved records for reporting purposes without modifying database information.
- **Database Administrator:** Manages existing database records but does not receive unrestricted superuser privileges.
- **Database Owner:** Owns database objects and is used for database setup and ownership-related operations.

5. Privilege Assignment

Privileges were assigned according to the project's documented access requirements.

5.1 Database and Schema Privileges

All three group roles were granted CONNECT permission on the ReserveIT database and USAGE permission on the public schema.

The CONNECT privilege was revoked from PUBLIC to prevent unrelated PostgreSQL accounts from connecting automatically.

The database was configured so that the application, reporter, and administrator accounts could connect through their assigned group roles.

5.2 Runtime Role

The app_runtime_grp role received table-specific privileges needed by the application.

The assigned permissions include:

- SELECT
- INSERT
- UPDATE
- DELETE

However, these permissions were limited to specific tables and columns according to the project's access requirements.

The runtime role was allowed to perform necessary operations such as:

- Registering and managing application users.
- Viewing department, staff, room, and material information.
- Creating and processing reservations.
- Updating reservation statuses.
- Managing approved room and material operations.
- Updating payment-related penalty information.

The runtime role was restricted from directly deleting sensitive records, including reservations, penalties, users, staff, and departments.

It was also restricted from changing sensitive fields such as password hashes, reservation details, penalty amounts, and reservation line items.

5.3 Read-Only Role

- The app_readonly_grp role was granted SELECT privileges on approved database tables.
- The reporter account was allowed to view information needed for reports, including reservation details, penalty information, and selected user and staff information.
- Sensitive information, including password hashes and email addresses, was restricted.
- The read-only role was not granted INSERT, UPDATE, DELETE, or table-creation privileges.

5.4 Administrative Role

- The app_admin_grp role was granted SELECT, INSERT, UPDATE, and DELETE privileges on the database tables.
- This allows the database administrator account to manage existing records and correct invalid data.
- However, the administrator was not granted unrestricted schema modification privileges, role creation privileges, or database creation privileges.

5.5 Database Ownership

- The database ownership was assigned to the separate reserveit_owner account.
- Existing tables were verified to be owned by reserveit_owner.
- The database owner account was separated from the application runtime and reporting accounts to avoid unnecessary ownership-level permissions.

6. Access Control Testing

The project's access-control tests were executed using:

```
python tests/access_test.py
```

The tests checked whether each database account could perform its intended operations and whether unauthorized operations were properly restricted.

6.1 Runtime Account — reserveit_app

The runtime account was tested against the application's required database operations.

The following access restrictions were verified:

- Allowed operations on approved tables and columns.
- Restricted access to sensitive fields.
- Denied changes to password hashes.
- Denied unauthorized changes to reservation details and penalty amounts.
- Denied deletion of protected records.
- Denied TRUNCATE operations.
- Denied table creation and deletion.
- Denied role and database creation.

The runtime account passed its assigned access-control checks.

6.2 Read-Only Account — reserveit_reporter

The reporter account was tested to verify that it could only read approved information.

The following were verified:

- SELECT access to approved reporting information.
- Restricted access to password hashes.
- Restricted access to email addresses.
- Denied INSERT operations.
- Denied UPDATE operations.
- Denied DELETE operations.
- Denied table creation.

The reporter account passed all 10 access-control checks.

6.3 Administrative Account — reserveit_dba

The administrative account was tested to verify its ability to manage existing database records.

The tests confirmed that the account could:

- Read database records.
- Correct invalid records.
- Delete records when necessary.

The account was also restricted from changing the database structure, creating or dropping tables, and creating PostgreSQL roles.

The administrative account passed all 7 access-control checks.

6.4 Database Owner — reserveit_owner

- The database owner account was tested to verify its ownership-related privileges.
- The tests confirmed that it could perform table-structure operations while remaining restricted from creating PostgreSQL roles or databases.
- The owner account passed all 3 access-control checks.

6.5 Additional Security Settings

The project also included checks for the following configurations:

- All four login accounts were verified as non-superusers.
- Application and reporting accounts were restricted from creating roles and databases.
- PUBLIC CONNECT was revoked from the ReserveIT database.

- Database tables were owned by reserveit_owner.
- The runtime, reporter, administrator, and owner accounts were tested separately.

7. Application and System Testing

Aside from database access-control tests, the project's application workflows were also tested.

7.1 Database Connection

- The application was successfully connected to the ReserveIT database using the reserveit_app runtime account.
- The database connection was checked through the project's management command.

7.2 Smoke Testing

The following command was executed:

```
python tests/smoke_test.py
```

The smoke tests covered the application's major workflows, including:

- Login and access control.
- User registration.
- Reservation rules.
- Reservation approval.
- Cancellation and completion.
- Penalty processing.
- Penalty threshold validation.
- Administrative and inventory operations.

Result: All 67 smoke tests passed.

7.3 Access-Control Testing Summary

- The access-control test script reported that all 48 checks passed.
- This confirms that the configured database accounts followed the tested access requirements of the ReserveIT project.

8. Issues and Limitations

During the database setup and access-control implementation, the following considerations were encountered.

8.1 Database Setup

- The project database was successfully initialized using the provided management commands.
- The database setup process included role configuration, schema initialization, and sample data insertion.

8.2 PostgreSQL Password Configuration

- The initial execution of the role setup command encountered a PostgreSQL authentication issue because of an incorrect password.
- The command was executed again using the correct PostgreSQL password, and the setup completed successfully.

8.3 Role and Privilege Configuration

- The database roles and their assigned privileges were configured according to the project's access requirements.
- The access-control test script was executed successfully, with all 48 checks passing.

8.4 Application Testing

- The application was successfully launched using the Flask development server.
- The login page was accessible locally at:
    - http://127.0.0.1:5000
- The database connection and major application workflows were also checked through the project's smoke tests.

8.5 Database Ownership

- The database tables were owned by reserveit_owner, keeping database ownership separate from the application runtime and reporting accounts.
- This setup helps prevent application accounts from receiving unnecessary ownership-level privileges.

9. Evidence and Repository

The SQL verification script, access-control documentation, and supporting screenshots are stored in the group's Car Rental Management System repository.

Repository links:

- **Main Repository:** https://github.com/123airabels/Car_Rental_Management_System
- **ReserveIT Access-Control Folder:** https://github.com/123airabels/Car_Rental_Management_System/tree/main/db-access-control/Group11

9.1 Evidence Screenshots

The evidence folder contains the following screenshots:

1. [01]ReserveIT_Access_Control_Tests.png
2. [01]ReserveIT_Access_Control_Tests_2.png
3. [02]ReserveIT_Smoke_Tests.png
4. [02]ReserveIT_Smoke_Tests_2.png
5. [03]ReserveIT_Login_Page.png
6. [04]ReserveIT_Database_Connection.png
7. [05]ReserveIT_Access_Verification/01.png
8. [05]ReserveIT_Access_Verification/02.png
9. [05]ReserveIT_Access_Verification/03.png

9.2 Supporting Files

The access-control folder also contains:

- Access_Control_Plan.md
- sql/reserveit_access_verification.sql
- evidence/

The access-control plan documents the project's role requirements and assigned permissions. The SQL verification script contains queries used to inspect and verify the configured access privileges.

The files and evidence screenshots were committed and pushed to the group's repository.

10. Verification Results

The access-control implementation was verified through the project's automated tests and additional PostgreSQL checks.

10.1 Runtime Account Verification

Operation	Expected Result	Actual Result
Approved SELECT operations	Allowed	Passed
Approved INSERT operations	Allowed	Passed
Approved UPDATE operations	Allowed	Passed
Protected field modifications	Denied	Passed
Unauthorized record deletions	Denied	Passed
TRUNCATE operations	Denied	Passed
CREATE/DROP TABLE	Denied	Passed
CREATE ROLE / DATABASE	Denied	Passed

10.2 Read-Only Account Verification

Operation	Expected Result	Actual Result
SELECT approved information	Allowed	Passed
SELECT password hashes	Denied	Passed
SELECT restricted email information	Denied	Passed
INSERT	Denied	Passed
UPDATE	Denied	Passed
DELETE	Denied	Passed
CREATE TABLE	Denied	Passed

10.3 Administrative Account Verification

Operation	Expected Result	Actual Result
Read database records	Allowed	Passed
Correct invalid records	Allowed	Passed
Delete existing records	Allowed	Passed
Modify database structure	Denied	Passed
CREATE/DROP TABLE	Denied	Passed
CREATE ROLE / DATABASE	Denied	Passed

10.4 Database Owner Verification

Operation	Expected Result	Actual Result
Create/change table structures	Allowed	Passed
CREATE ROLE	Denied	Passed
CREATE DATABASE	Denied	Passed

10.5 Overall Test Results

Test Category	Number of Checks	Result
Access-control tests	48	All Passed
Smoke tests	67	All Passed
Total	**115**	**All Passed**

The test results confirmed that the configured accounts could perform their intended operations while unauthorized operations were restricted.

11. Issues and Fixes

The following issues and observations were recorded during the implementation.

Issue	Action Taken / Resolution	Status
PostgreSQL authentication failed during the initial role setup.	Re-executed the setup command using the correct PostgreSQL password.	Resolved
Application database access needed to be checked using the intended runtime account.	Verified the connection using reserveit_app.	Resolved
Application and reporting accounts needed restrictions on sensitive information.	Configured table-level and column-level privileges according to the access requirements.	Verified through tests
Application accounts needed to be prevented from performing database administration operations.	Restricted role creation, database creation, and schema modification operations.	Verified through tests
Database ownership needed to be separated from application access.	Used reserveit_owner as the owner account.	Verified

12. Feedback to the Owning Group

The following observations and recommendations were identified while setting up and testing ReserveIT.

Area	Observation	Recommendation
Database Setup	The database setup commands successfully initialized the database and sample records.	Continue providing clear setup instructions and required environment configurations.
Database Privileges	Separate runtime, readonly, and admin roles were configured.	Maintain least-privilege access and review permissions whenever new features are added.

Sensitive Information	The reporter account was restricted from accessing password hashes and email addresses.	Continue protecting sensitive user information through column-level restrictions.
Database Ownership	The database tables were owned by a separate owner account.	Keep the owner account separate from normal application and reporting accounts.
Application Testing	All 67 smoke tests passed.	Continue testing reservation workflows and edge cases after future updates.
Access-Control Testing	All 48 access-control checks passed.	Maintain automated access tests whenever database privileges or application requirements change.
Documentation	The project provided access requirements and user-role documentation.	Keep the database setup and permission requirements updated when the system changes.

13. AI Usage Declaration

AI tools, including ChatGPT, were used as assistance during the ReserveIT database setup and access-control implementation.

The AI assistance was used for the following tasks:

- Understanding PostgreSQL roles, memberships, and privileges.
- Reviewing the project's database access requirements.
- Explaining PostgreSQL permission and authentication errors.
- Preparing and reviewing SQL verification queries.
- Understanding the expected results of allowed and denied operations.
- Organizing the access-control documentation.

All database setup commands and access-control tests were executed and checked using the local PostgreSQL environment. The results recorded in this document were based on the actual test outputs and available screenshots.

AI was used as a support tool. The implementation and recorded results were reviewed against the project's documentation and actual testing results.

14. Conclusion

The ReserveIT database was successfully initialized and configured for Activity 4 access-control testing using PostgreSQL 18.6.

Three NOLOGIN group roles and their corresponding login accounts were configured for administrative, application runtime, and read-only responsibilities. A separate database owner account was also used to manage database ownership.

Privileges were assigned based on the principle of least privilege. The runtime account was limited to the database operations required by the application, the reporter account was restricted to approved information, and the administrator account was allowed to manage existing records without receiving unrestricted database structure or superuser privileges.

Verification was performed through 48 access-control checks and 67 smoke tests. All 115 checks passed.

The project was also successfully launched locally, and its database connection and major application workflows were tested.

The access-control plan, SQL verification script, and supporting screenshots were saved in the group's repository.

Overall, the implementation provided practical experience in PostgreSQL role management, privilege assignment, database ownership, access restrictions, and security verification.